MCU – SoC – FPGA

Đánh đổi giữa hiệu năng, chi phí và độ linh hoạt

GVHD: …
Nhóm: … (điền họ tên – MSSV)

Môn CO3053 Hệ thống nhúng · HK261
Trường Đại học Bách khoa – ĐHQG-HCM

Tháng 11/2026

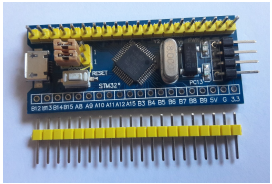
VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Câu hỏi của bài

Khi nào nên chọn **MCU**, **FPGA** hay **SoC**?

- Xét theo 3 tiêu chí cốt lõi: **hiệu năng** (thông lượng, độ trễ), **chi phí** (chip, board, công suất), và **độ linh hoạt** (hệ điều hành, tái cấu hình).
- *Không* nhằm chứng minh nền tảng nào "tốt nhất", vì đó là mục tiêu thiên vị mà phương pháp đánh giá hiệu năng (pe2) cảnh báo.
- Mọi con số trong bài đều mang một nhãn minh bạch xác định nguồn gốc (pe3):
  [đo] [mô phỏng] [ước lượng công cụ] [tính] [nguồn]

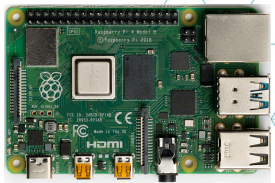
Ba kiến trúc: Căn nguyên của sự đánh đổi



Popolon, CC BY-SA 4.0



S03311251, CC BY-SA 4.0



Laserlicht, CC BY-SA 4.0

MCU

Lõi nhỏ tuần tự (Cortex-M), flash và SRAM trên chip
Không có MMU, không chạy Linux
Khởi động vài ms, ngắt dưới 1 µs
Dòng nghỉ cỡ nA, giá vài USD

FPGA

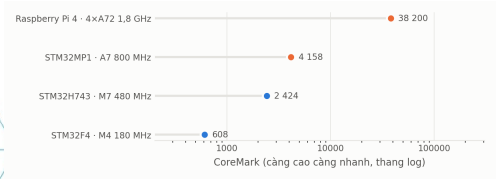
Mảng logic song song (LUT, DSP, BRAM nối tùy ý)
Thuật toán trải phẳng ra phần cứng: **1 mẫu/chu kỳ**
Độ trễ cố định cỡ nano-giây
Đổi lại: giá cao, lập trình HDL

SoC

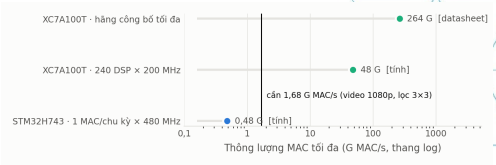
Lõi CPU lớn (Cortex-A) có MMU, cache, GPU, codec
Chạy **Full Linux**: driver, mạng, AI có sẵn
Đổi lại: độ trễ không tiền định
Công suất nghỉ vài Watt

Khi cần cả hệ điều hành lẫn timing cứng: tích hợp CPU + FPGA trên cùng một chip (**phương án lai Zynq**)

Năng lực tính toán: CPU tuần tự vs. Dòng dữ liệu song song



Dữ liệu: ST overview, OpenBenchmarking (Pi 4). Nhóm vẽ lại.



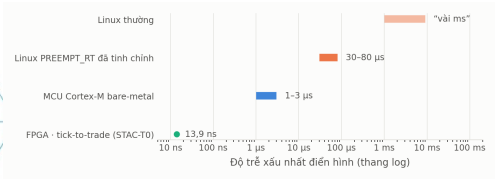
Dữ liệu: Lee & Seshia mục 8.1; datasheet XA Artix-7. Nhóm vẽ lại.

CPU tuần tự: **SoC** Pi 4 mạnh gấp ≈**16×** **MCU** H743. FPGA không chạy lệnh tuần tự.

Lọc video 1080p: **MCU** mạnh nhất chỉ đạt ≈29% nhu cầu; **FPGA** chỉ dùng ≈3,5% DSP gánh trọn 100% dòng dữ liệu.

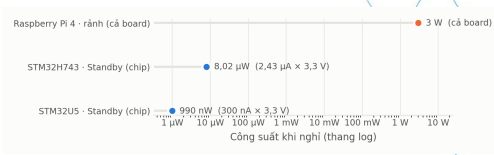
Arucu & Iliev (2025): FIR trên FPGA nhanh hơn Cortex-A9 ≈**27×** và tốn ít năng lượng hơn ≈**95–190×** [tính].

Độ trễ xấu nhất và Công suất khi nghỉ theo tài liệu



Dữ liệu: AC6 (MCU, Linux); A-Team Insight, STAC-T0 (FPGA). Nhóm vẽ lại.

Độ trễ xấu nhất: Mỗi nền tảng cách nhau 1–2 bậc độ lớn (FPGA: ns → MCU: µs → Linux RT: 30–80 µs → Linux thường: vài ms).



Dữ liệu: ST H743, Arrow (U5), TechFeed (Pi 4). Nhóm vẽ lại.

Công suất nghỉ: MCU Standby 300 nA vs SoC 3 W (Δ ≈**3 triệu lần**). Giá: MCU £9–12 · SoC $35 · FPGA €144.

Phương án lai CPU + FPGA: ADAS và điều khiển công nghiệp

- Bài toán cần **cả** hệ điều hành phong phú **lẫn** timing cứng hoặc xử lý song song tốc độ cao.
- **Zynq UltraScale+:** gom trên một chip 4×A53 chạy Linux, 2×R5 lockstep chạy real-time, và fabric FPGA.
- **Lecture 3, Option #2:** CPU lo tính toán mềm dẻo, FPGA giữ nhịp cơ cấu chấp hành (actuator) độc lập kể cả khi CPU bận.
- FIR trên SoC FPGA+ARM nhanh gấp **8,86×** ARM đơn, giá/hiệu năng tốt nhất trong các cấu hình được thử [nguồn].



S03311251, CC BY-SA 4.0

Zynq UltraScale+: Arm + FPGA trên một chip

Cái giá: chi phí thiết kế và giá chip cao hơn, độ linh hoạt phần mềm thấp hơn so với CPU đơn.

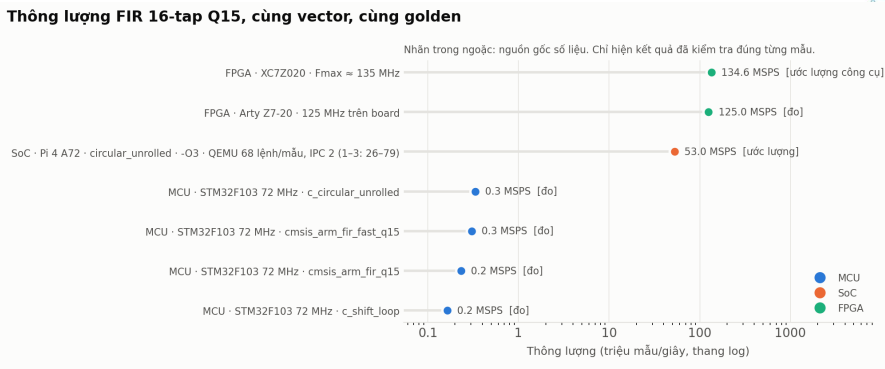
Thí nghiệm: một bài toán, ba nền tảng

FIR 16-tap Q15, 1024 mẫu: bậc thang, giá trị biên, mẫu ép bão hòa, mẫu ngẫu nhiên (seed 3053)
Mọi nền tảng phải khớp **từng mẫu (0 sai lệch)** với golden model viết bằng Python

Nền tảng	Phần cứng	Phương pháp đo đạc thực nghiệm
MCU	STM32F103C6 @ 72 MHz (Cortex-M3)	Đo thật: 4 bản C/CMSIS, đếm chu kỳ bằng DWT; ngắt TIM2 trong 3 kịch bản tải
FPGA	Arty Z7-20 @ 125 MHz (XC7Z020 PL)	Đo thật: Pipeline 7 tầng thuần LUT (0 DSP); tự kiểm ngay trên chip qua JTAG
SoC	Cortex-A72 @ 1,8 GHz (QEMU ARMv8)	Mô phỏng: Chu kỳ lệnh QEMU AArch64, ước lượng thông lượng IPC 1–3; trễ từ AC6

Cortex-M3 không có lệnh SIMD (SMLAD), nên CMSIS-DSP trên M3 không được tăng tốc kiểu SIMD như trên M4/M7.

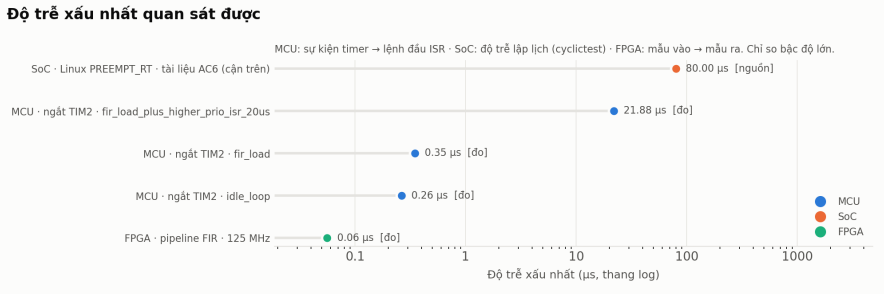
Kết quả: thông lượng — FPGA vượt trội 375×



FPGA: **125,0 MSPS** (1,00 mẫu/chu kỳ @ 125 MHz) [đo] **SoC**: 53,0 MSPS [ước lượng; IPC 1–3: 26–79] **MCU**: **0,33 MSPS** [đo]

Phát hiện kỹ thuật: **FPGA** nhanh gấp **375×** bản tối ưu của MCU (và gấp 763× bản C cơ bản). Do thiếu SIMD, MCU chỉ đạt **7%** so với giả thiết lý thuyết 1 MAC/chu kỳ.

Kết quả: độ trễ xấu nhất và tính tiền định (Jitter)



FPGA pipeline: **56 ns** (7 chu kỳ @ 125 MHz), Jitter = **0,00 ns** [đo]
MCU ngắt khi rảnh: 264 ns · khi chạy FIR: 264–347 ns [đo]
SoC Linux thường: vài ms [nguồn: AC6] · PREEMPT_RT: 30–80 µs [nguồn: AC6]

Thí nghiệm phản biện đắt giá: Khi thêm ngắt ưu tiên cao hơn chen ngang 20 µs, độ trễ ngắt MCU vọt lên tới **21,9 µs**. Worst-case của MCU do **kiến trúc phần mềm** quyết định, không phải chỉ do chip.

Thực nghiệm đặc biệt: FPGA thuần Soft-Logic (0 DSP)

- **Câu hỏi thực tế:** *"Nếu các khối DSP48E1 bị chiếm hết cho tác vụ khác, FPGA có lọc được không?"*
- Nhóm ép RTL chạy thuần soft-logic: `(* use_dsp = "no" *)`, tổng hợp Vivado `-max_dsp 0`.
- 16 bộ nhân song song và cây cộng 4 tầng 34-bit dựng hoàn toàn bằng **Slice LUTs + CARRY4**.
- Đóng timing an toàn: WNS $= +$**0**,**326** ns ở 125 MHz ($F_{\text{max}} \approx 134{,}6$ MHz).
- Tiêu thụ: **0,124 W** ($\approx$ **0**,**99** nJ/mẫu), thấp hơn cả bản dùng DSP (0,129 W).

Tài nguyên	Đã dùng	Chiếm (%)
DSP48E1	**0 / 220**	**0,00%**
Slice LUTs	**1 672 / 53 200**	**3,14%**
Thanh ghi FF	**1 046 / 106 400**	**0,98%**
Block RAM	2 / 140	1,43%

Ý nghĩa: Chỉ cần **3,1% logic thường** của XC7Z020 đã đạt 125 MSPS, dành trọn 220 khối DSP cho tác vụ khác.

Kiểm chứng chéo với tài liệu

Câu hỏi kiểm chứng	Nhóm đo được	Tài liệu nói
MCU đạt được bao nhiêu so với “1 MAC/chu kỳ”?	**7%** (216,3 chu kỳ/mẫu)	PLAN giả định 100% (≈0,48 G MAC/s trên H743)
FPGA nhanh hơn MCU / CPU bao nhiêu lần?	**375×** (vs MCU); 2,4× (vs Pi 4)	≈27× so với Cortex-A9 (Arucu & Iliev)
Độ trễ ngắt MCU bare-metal	264–347 ns (rảnh/tải)	1–3 µs (AC6)
Worst-case MCU khi có ngắt lồng nhau 20 µs	**21,9 µs**	Thiết kế phần mềm quyết định worst-case
Tăng từ 16 lên 64 tap thì sao?	MCU chậm đi tuyến tính 4× (80 kS/s)	FPGA giữ nguyên 125 MSPS (cách biệt >1 500×)

Quan sát phụ: Một lõi CPU x86 desktop lọc được 140 MSPS. FPGA không thắng nhờ tốc độ thô ở bài nhỏ, mà nhờ **thông lượng giữ nguyên khi tăng tap**, **độ trễ cố định** và **công suất thấp**.

Bản đồ ứng dụng: Mỗi nền tảng thống trị ở đâu?

MCU: ECU ô tô, động cơ, cảm biến pin
• AURIX lockstep đạt **ASIL-D**, AEC-Q100 150 °C
• Khởi động vài ms, ngắt < 1 µs, giá vài USD
• *Vì sao chọn:* SoC chậm khởi động & không xác định; FPGA đắt.

SoC: Router Wi-Fi, điện thoại, camera IP
• Qualcomm IPQ: Linux + khối xử lý gói 37,5 Mpps
• Snapdragon: CPU + GPU + NPU + 5G modem 3 nm
• *Vì sao chọn:* MCU không có MMU; FPGA tốn công HDL.

FPGA: Siêu âm, 5G radio, tạo mẫu ASIC, HFT
• Siêu âm beamforming 128–1024 kênh ADC song song
• Trạm 5G O-RAN nạp lại được; HFT tick-to-trade 13,9 ns
• *Vì sao chọn:* MCU thiếu sức; SoC trễ không xác định.

Lai: ADAS xe tự hành, Robot công nghiệp
• Zynq: Cortex-A (Linux/AI) + FPGA (bảo vệ timing cơ cấu)
• *Vì sao chọn:* Một nền tảng đơn lẻ không thể thỏa mãn cả hai.

Ma trận quyết định chọn nền tảng

Nếu bài toán…	Chọn	Vì
Chạy pin nhiều năm, tác vụ nhẹ	**MCU**	300 nA ở Standby; hơn 10 năm trên pin CR2032
Deadline vài chục µs, cần an toàn cứng	**MCU**	Ngắt dưới 1 µs; tiêu chuẩn ASIL-D
Cần Linux, camera, mạng, GUI; timing mềm	**SoC**	CoreMark gấp ≈16× H743; driver có sẵn
Trên ≈0,5 G MAC/s liên tục, I/O streaming cao	**FPGA** / Lai	1 mẫu/chu kỳ (E2), ≈44–48 G MAC/s
Độ trễ cố định cỡ nano-giây	**FPGA**	Pipeline 56 ns (0 jitter), HFT 13,9 ns
Cần cả OS phong phú lẫn timing cứng ngắc	Lai	Zynq, Lecture 3 Option #2

Quy luật cốt lõi: Muốn timing và thông lượng cao hơn thì phải trả bằng **giá tiền cao hơn** và **độ linh hoạt phần mềm thấp hơn**.

“Không có nền tảng nào là tốt nhất,

chỉ có nền tảng **ĐỦ** cho con số quyết định của bài toán, với chi phí thấp nhất.”

— Thông điệp cốt lõi của đề tài

MCU	Thắng về **giá**, **công suất nghỉ** (nA) và **chứng nhận an toàn**. Giới hạn ở tính toán tuần tự.
SoC	Thắng khi cần **hệ điều hành Linux**, mạng và đồ họa. Giới hạn ở độ trễ không tiền định.
FPGA	Thắng khi cần **thông lượng song song** và **độ trễ nano-giây**. Giới hạn ở giá và độ phức tạp HDL.

Cảm ơn Thầy Cô và các bạn

Nhóm sẵn sàng nhận câu hỏi phản biện

CO3053 Hệ thống nhúng · HK261 · Đề tài 09

Phụ lục: Chuẩn bị cho câu hỏi phản biện

- **“MCU đã tối ưu chưa? Sao không dùng M4/M7?”** Có bản CMSIS-DSP. M3 đại diện dòng giá rẻ không có SIMD. Cortex-M4/M7 có SIMD (`SMLAD`) nhanh hơn 3–4× (50 chu kỳ/mẫu), nhưng vẫn chậm hơn FPGA hàng chục đến hàng trăm lần vì FPGA nhân 16 tap cùng lúc.
- **“Jitter của FPGA bằng 0 thật à?”** Đúng ở mức chu kỳ (cycle-accurate): tiền định tuyệt đối 7 chu kỳ. Ở mức vật lý tín hiệu, chỉ có phase jitter của thạch anh (cỡ pico-giây), không đáng kể so với mức nano/micro-giây.
- **“Công suất FPGA đo bằng gì?”** Vivado `report_power` (vector-less) mang nhãn [ước lượng công cụ], gồm cả công suất tĩnh 0,106 W.
- **“Tăng lên 64 tap thì sao?”** MCU chậm đi 4× (80 kS/s). FPGA mở rộng cây cộng thêm 2 tầng (trễ tăng lên 9 chu kỳ = 72 ns), nhưng **thông lượng vẫn giữ nguyên 125 MSPS**. Cách biệt tăng từ 375× lên >1 500×.

Phụ lục: Nguồn tài liệu & Bản quyền hình ảnh

Tài liệu tham khảo chính:

- ST: STM32H7/F4/MP1 overview, STM32H743; OpenBenchmarking: Raspberry Pi 4 CoreMark
- Arucu & Iliev, J. Low Power Electron. Appl., 2025 (FIR trên FPGA, GPU, CPU)
- AC6: Độ trễ Linux PREEMPT_RT; A-Team Insight: Exegy/AMD STAC-T0 (13,9 ns tick-to-trade)
- Embedded World 02/2026; Automotive World (AURIX TC3xx); Future Electronics (NXP S32K)
- Macnica, CNX Software (Qualcomm IPQ); GSMArena (Snapdragon 8 Elite)
- Kou et al., 2023 (Beamforming siêu âm FPGA); Synopsys (HAPS ASIC prototyping)
- Lee & Seshia, *Introduction to Embedded Systems*, mục 8.1–8.2; Lecture 2, Lecture 3.

Hình ảnh: Wikimedia Commons (giấy phép mở Creative Commons ghi chú bên dưới mỗi ảnh). Biểu đồ dựng từ số liệu công bố.